

Why local variables need to be stored in stack to make them local

Writing a function in C (collee)

```
int Leaf (int g, int h, int i, int j){
    int f;
    f=(g+h)-(i+j);
    return j;
}
```

Testing the above function (caller)

```
int main(void){
    //we should be able to name our variables without worrying that other functions
    // may be using same names since variables declared inside a function are local to
    //the function (main of course is also a function)
    int f=7, int m=10, int l=11, int z=20;

    ...
    int g= Leaf(f,m,l,z);
    f= f+g;
    printf(f);
    return 0;
}
```

We are now going to translate the above code to RISC-V assembly and going to start translating the caller/testing function first (since it is easier), and I'm going to simplify this function since it is also only for testing, and will not use the stack for it to avoid confusion (but you should know that all variables declared in main are local to main and should definitely also go on the stack)

Main:

```
Li x18, 7          //Note that since f,m,l,z are local var I should use saved reg so f->x18
                   //m->x19,l->x20,z->x21

Li x19,10
L1 x20,11
L1 x21,20
...
//According to function execution I have to know do two things, first move values
//into input register and second transfer control to the function call
Mov x10,x18
Mov x11,x19
Mov x12,x20
Mov x13,x21
Jal leaf -----□ saves ra=pc+4 (the address of the pink instruction) and then pc=leaf jumps to
                    function

//read result from function call
Add x18, x18,x10 //NOTE: the expected value is (x18=7) + (x10=17-31=-14)-□ -7
                //from Leaf we should not be able to modify the value of variable f at register x18
//Print x18 and exit (I don't want to do this in assembly, so assume we can print a value here)
```

Leaf:

//allocate local variables into the stack, again, we should be able to use whatever name we want for

```

//local variables without having to find what names are used by other functions
sub sp, sp,4

sw x18,0(sp) □ I'm going to use the register x18 for my local variable f and by saving it on stack
                I do not have to worry if it is being used somewhere else, (right now on stack it will
be
                Saved that x18 is valued 7

//body of the function read the input registers and add
Add x5,x10,x11 //save on a temporary register (not expected to be preserved) x5 is 7+10=17
Add x18 ,x12,x13 //now x18 should have the value (11+20=31) x18=31
Sub x18, x15,x8 //now x18 should have now value 31-17=-14. x18=-14

//move result into result register
Mov x10,x18 //x10 contains -14

//restore the stack
//if we do not do this the value of x18 is -14 and when returning to main the value for f in main
//will change to -14. SO do not forget to restore stack
Lw x18,0(sp) -□ x18 =7 (read from stack and restored to initial value
Add,sp,sp,4

//return to caller
Ret -> pseudo instruction for pc=ra so after this instruction the pc will point to inst in pink on main

```

What happens if the number of input parameters is more than the registers for input parameters?

What happens if the function itself calls another function or is recursive?